

Contents

Qubes Secure SDLC / DevSecOps Lab — итоговый технический отчет	1
1. Executive Summary	2
2. Цель и критерии успеха	2
3. Архитектура Qubes и модель изоляции	2
4. Репозиторий и evidence layout	3
5. Уязвимый baseline	3
6. Security tooling	3
7. Результаты before/after	4
8. Policy gate	5
9. Remediation	5
10. Evidence map	5
11. Почему отдельные VM имеют смысл	6
12. Limitations	7
13. Публикационная готовность	7
14. Как защищать проект на интервью	7
15. Финальное заключение	7
16. Control mapping	8
17. Tool-by-tool interpretation	8
Simgrep	8
Bandit	8
pip-audit	8
Gitleaks	9
Checkov	9
Trivy	9
Syft and Gype	9
18. Evidence integrity model	9
19. Release decision logic	9
20. Reviewer checklist	10
21. Interview Q&A	10
22. Future improvements	11
23. Final reviewer statement	11
24. Детальная хронология фаз	11
25. Risk acceptance record	12
26. Reproduction guide для reviewer	13
27. Что именно улучшено в финальной редакции	13
28. Почему кейс сильный	14
29. Expected GitHub reviewer impression	14

Qubes Secure SDLC / DevSecOps Lab — итоговый технический отчет

Дата финальной редакции: 2026-05-09 07:34:58 UTC

Статус: publication-ready

Основной результат: уязвимый baseline воспроизводимо блокируется policy gate,

исправленная версия воспроизводимо проходит gate.

1. Executive Summary

Проект демонстрирует полный Secure SDLC / DevSecOps кейс в стиле SOC evidence workflow. В Qubes OS был разделен рабочий контур: разработка и документация выполнялись в dev-workbench, security tooling и сканирования — в security-runner, перенос evidence выполнялся вручную через Qubes core workflow. Это не автоматизация ради автоматизации, а контролируемый изолированный процесс, где каждый результат сохраняется как артефакт.

Цель проекта достигнута: создан уязвимый baseline, собраны отчеты SAST, SCA, secrets, IaC, container/config и SBOM, зафиксирован провал policy gate, затем выполнена remediation, повторные проверки показали отсутствие gate-blocking условий, а итоговый policy gate получил статус PASSED.

Ключевая ценность проекта не только в наличии уязвимого и исправленного кода. Сильная сторона кейса — в доказуемости. Для каждой фазы есть command output, scan report, summary, tree, git evidence и публикационный контроль.

2. Цель и критерии успеха

Главная цель: показать, как Secure SDLC pipeline может выглядеть в изолированной Qubes-среде с SOC-подходом к доказательствам.

Критерии успеха:

1. Уязвимая версия содержит намеренные дефекты приложения, зависимостей, секретов, Dockerfile и IaC.
2. Security tooling выполняется в отдельной AppVM, а не смешивается с рабочей VM.
3. Все проверки сохраняют machine-readable отчеты.
4. Policy gate на уязвимой версии завершается FAILED.
5. Remediation устраняет gate-blocking риски.
6. Policy gate на исправленной версии завершается PASSED.
7. Evidence pack содержит дерево проекта, hash manifest, отчеты, SBOM, итоговый отчет и презентацию.
8. Репозиторий готов к публичной публикации без временных cache и backup-директорий.

3. Архитектура Qubes и модель изоляции

Проект использует практическое разделение ролей:

- dev-workbench — рабочая VM для репозитория, документации, Git, отчетов и evidence inventory.
- security-runner — VM для запуска Semgrep, Bandit, pip-audit, Gitleaks, Checkov, Trivy, Syft и Grype.

- TemplateVM — база для пакетов системы, но пользовательские security tools устанавливались в AppVM, чтобы не ожидать переноса содержимого ~/.local из template.
- Передача evidence выполнялась через Qubes copy workflow. Это сохраняет границу доверия между VM и делает перенос данных явным действием.

Такой подход важен для портфолио: он показывает не просто знание инструментов, а понимание того, почему безопасность требует разделения рабочих зон, минимизации доверия и воспроизводимого evidence trail.

4. Репозиторий и evidence layout

Проект оформлен как case study, а не как набор случайных команд. Основные зоны:

- app/vulnerable-version — намеренно уязвимое приложение.
- app/fixed-version — исправленная версия.
- docker — vulnerable/fixed Dockerfile.
- iac/vulnerable и iac/fixed — Terraform examples.
- scripts — подготовленные control scripts и policy gate.
- evidence/command_outputs — полный журнал ключевых команд.
- evidence/scan_reports — JSON-отчеты security tools.
- evidence/sbom — SBOM CycloneDX от Syft.
- evidence/before_after — matrices и summaries.
- reports — итоговые RU/EN отчеты.
- presentation — storyboard, slide source и HTML-презентация.
- docs/ru и docs/en — фазовая документация.

5. Уязвимый baseline

Уязвимая версия нужна не для эксплуатации реальных систем, а как учебный контролируемый объект. Она содержит дефекты, которые типично встречаются в DevSecOps pipeline:

- insecure application patterns;
- dependency vulnerabilities;
- lab secrets;
- weak IaC security group model;
- insecure Dockerfile practices;
- отсутствие passing policy gate.

Baseline зафиксирован отдельной фазой и тегом, затем использован как источник для сравнения before/after.

6. Security tooling

В проекте использованы инструменты нескольких классов:

- Semgrep — SAST и multi-language правила.
- Bandit — Python security scanning.

- pip-audit — Python dependency SCA.
- Gitleaks — secrets detection.
- Checkov — IaC scanning.
- Trivy — filesystem dependency scan, IaC/config scan и Dockerfile scan.
- Syft — SBOM generation.
- Gype — SBOM vulnerability scan.

Это покрывает не один тип проверки, а полный DevSecOps control plane: code, dependencies, secrets, IaC, container configuration, SBOM и policy gate.

7. Результаты before/after

Контроль	Уязвимая версия	Исправленная версия	Интерпретация
Policy gate	FAILED	PASSED	FAILED baseline -> PASSED fixed release
Semgrep findings	10	0	all Semgrep findings removed
Semgrep ERROR findings	5	0	blocking severity removed
Bandit findings	7	0	all Bandit findings removed
Bandit HIGH findings	2	0	no HIGH issues in fixed app
pip-audit vulnerabilities	24	4	residual non-blocking dependency findings documented
Gitleaks findings	2	0	lab secrets removed from fixed app
Checkov failed checks	4	1	public ingress removed; residual unattached SG check documented
Checkov public ingress	4	0	0.0.0.0/0 removed
Trivy fs vulnerabilities	17	4	dependency risk reduced
Trivy fs HIGH/CRITICAL	6	0	no gate-blocking high/critical risk
Gype SBOM matches	17	4	SBOM scan retained for transparency

Контроль	Уязвимая версия	Исправленная версия	Интерпретация
Grype HIGH/CRITICAL	6	0	no high/critical SBOM blockers
Syft SBOM components	5	3	smaller dependency surface

Вывод по таблице: фиксированная версия не является “идеально пустой” с точки зрения любых сканеров, потому что часть инструментов возвращает residual non-blocking findings. Это нормально для зрелого security engineering подхода: gate должен блокировать согласованные критичные условия, а не любой шум. Остаточные findings сохранены и объяснены, а не скрыты.

8. Policy gate

Policy gate — центральная контрольная точка проекта. Он переводит набор отчетов в понятный engineering decision:

- уязвимый baseline: POLICY_GATE_STATUS=FAILED;
- исправленная версия: POLICY_GATE_STATUS=PASSED.

Gate-relevant условия включают blocking findings по SAST, Bandit HIGH, secrets, публичный ingress, HIGH/CRITICAL container/dependency risks и SBOM HIGH/CRITICAL matches. После remediation все blocking counters стали нулевыми.

9. Remediation

Remediation была выполнена как отдельная engineering-фаза, а не как ручное удаление отчетов. Исправления разделены по зонам:

- application code: безопасная обработка inputs и удаление lab secrets из fixed version;
- dependencies: сокращение и обновление dependency surface;
- IaC: удаление public ingress 0.0.0.0/0;
- Dockerfile: hardening и non-root подход;
- evidence: remediation matrix и повторные сканы.

10. Evidence map

Фаза	Что доказывает	Артефакт	Evidence
00-03	Базовая подготовка	docs/ru/03_qubes_vm_evidence_prep_and_outputs/DEV docs/ru/04_template_appvm_preparation.md	

Фаза	Что доказывает	Артефакт	Evidence
04	Уязвимый baseline	app/vulnerable-version, docker/vulnerable.Dockerfile, iac/vulnerable/main.tf	evidence/command_outputs/DEV
05	Security tooling	docs/ru/06_security_tooling_and_evidence_installation_and	evidence/command_outputs/DEV
06	Vulnerable scans	Semgrep, Bandit, pip-audit, Gitleaks, Checkov, Trivy, Syft, Gype	evidence/command_outputs/DEV
07	Failed policy gate	scripts/08_policy_gates.sh	evidence/command_outputs/DEV
08	Remediation	app/fixed-version, docker/fixed.Dockerfile, iac/fixed/main.tf	evidence/before_after/DEVSECOP
09	Fixed scans + passed gate	fixed scan reports and SBOM	evidence/command_outputs/DEV
10	Evidence pack	EVIDENCE_INVENTORY.md, SHA256 manifest, project trees	evidence/DEVSECOPS_10_SHA25
11	Technical report	reports/devsecops_lab_report.md and EN version	evidence/command_outputs/DEV
12	Presentation	presentation/devsecops_evidence_defense_and_outputs/DEV and HTML	evidence/command_outputs/DEV
13	Publication cleanup	README, release notes, publication check	evidence/command_outputs/DEV

11. Почему отдельные VM имеют смысл

Даже при ручном переносе evidence отдельные VM полезны:

1. Tooling VM может быть подключена к интернету для скачивания DB и rules, а рабочая VM остается более контролируемой.
2. Результаты security tools не смешиваются с исходной разработческой средой.
3. Перенос evidence становится осознанным boundary crossing.
4. Компрометация одного tooling процесса не равна компрометации всей рабочей зоны.
5. Такая модель похожа на production security practice: build, scan, evidence и release имеют разные доверенные зоны.

Ручной перенос через Qubes GUI не ослабляет кейс. Наоборот, он показывает, что проект учитывает реальную operational security модель Qubes.

12. Limitations

Проект является лабораторным и не претендует на production deployment. Ограничения:

- нет реального CI runner в GitHub Actions с secrets и protected branches;
- security tools запускались вручную в Qubes AppVM;
- IaC examples являются демонстрационными;
- residual non-blocking findings сохранены для прозрачности;
- SBOM и vulnerability DB зависят от состояния tool databases на момент запуска;
- скриншоты являются дополнительным evidence, а не единственным источником доказательств.

13. Публикационная готовность

Перед публикацией выполнена cleanup-проверка:

- удалены __rusache__;
- удалены backup-директории;
- добавлены semantic screenshot names;
- обновлен report pack;
- пересобраны evidence inventory и hash manifest;
- добавлены RU/EN report PDFs и HTML versions;
- создан final publication review.

14. Как защищать проект на интервью

Короткая defense story:

1. Я построил Secure SDLC lab в Qubes OS с разделением разработки и security scanning.
2. Сначала создал намеренно уязвимый baseline.
3. Запустил набор инструментов: SAST, SCA, secrets, IaC, Trivy, SBOM, Grype.
4. Policy gate доказуемо заблокировал baseline.
5. Затем я сделал remediation и повторил проверки.
6. Исправленная версия прошла policy gate.
7. Все команды, отчеты, SBOM, trees, checksums, docs и презентация сохранены как evidence.

15. Финальное заключение

Цель задания достигнута. Проект показывает полный цикл: design -> vulnerable baseline -> security scans -> failed gate -> remediation -> fixed scans -> passed gate -> evidence pack -> report -> presentation -> publication readiness.

Это сильный портфолио-кейс для DevSecOps, Secure SDLC, AppSec, security automation и Qubes-based security workflow.

16. Control mapping

Проект можно читать как набор security controls, а не только как учебный репозиторий.

Control area	Реализация в проекте	Evidence
Source security	Semgrep и Bandit для application code	evidence/scan_reports/semgrep, evidence/scan_reports/bandit
Dependency security	pip-audit, Trivy fs, Gype over SBOM	evidence/scan_reports/pip-audit, evidence/scan_reports/trivy, evidence/scan_reports/gype
Secret hygiene	Gitleaks и отдельный .gitleaks.toml	evidence/scan_reports/gitleaks
IaC security	Checkov и Trivy IaC	evidence/scan_reports/checkov, evidence/scan_reports/trivy
Container hardening	Trivy Dockerfile scan	evidence/scan_reports/trivy
Software transparency	Syft CycloneDX SBOM	evidence/sbom
Release decision	scripts/08_policy_gate.sh	Phase 7 and Phase 9 command outputs
Evidence integrity	SHA256 manifest	evidence/DEVSECOPS_10_SHA256SUMS.txt

17. Tool-by-tool interpretation

Semgrep

Semgrep отвечает за раннее обнаружение insecure coding patterns. В baseline он показывает, что приложение действительно содержит security-relevant findings. В fixed version количество Semgrep findings равно 0, а ERROR findings равно 0. Это важно, потому что gate ориентируется на blocking severity, а не на субъективное ощущение “код стал лучше”.

Bandit

Bandit добавляет Python-specific взгляд на безопасность. В baseline были findings, включая HIGH-level findings. В fixed version Bandit findings стали 0. Это демонстрирует, что remediation коснулась не только инфраструктуры, но и application layer.

pip-audit

pip-audit показывает dependency risk. В fixed version осталось 4 total vulnerabilities. Они сохранены как residual non-blocking evidence. Это зрелая позиция:

реальные проекты редко имеют абсолютно пустой dependency report, поэтому важно отделять release-blocking risk от backlog risk.

Gitleaks

Gitleaks подтверждает, что fixed version не содержит lab secret findings. Для проекта это особенно важно, потому что secret hygiene является одной из самых частых причин инцидентов в реальных репозиториях.

Checkov

Checkov на fixed IaC возвращает 1 failed check. Главное gate-relevant условие — отсутствие public ingress 0.0.0.0/0 — выполнено: fixed public ingress occurrences равно 0. Остаточный check связан с демонстрационным характером Terraform example и зафиксирован явно.

Trivy

Trivy используется сразу в нескольких режимах: filesystem vulnerabilities, IaC/misconfiguration scanning и Dockerfile scanning. В fixed version HIGH/CRITICAL filesystem vulnerabilities равны 0, а Dockerfile HIGH/CRITICAL misconfigurations равны 0. Это подтверждает, что gate-blocking risk снят.

Syft and Gype

Syft создает SBOM, Gype сканирует SBOM. Эта связка важна, потому что она показывает supply chain transparency: проект не только сканирует исходники, но и фиксирует состав компонентов.

18. Evidence integrity model

Evidence model построена вокруг трех принципов.

Первый принцип — каждая важная команда сохраняет stdout/stderr в evidence/command_outputs. Это позволяет проверить не только итоговый JSON, но и контекст запуска: дату, kube, target, return code, counters.

Второй принцип — machine-readable reports сохраняются отдельно от human-readable summaries. JSON-файлы можно пересчитать через jq, а Markdown/CSV summaries можно читать вручную.

Третий принцип — финальный SHA256 manifest позволяет увидеть, менялись ли файлы после сборки evidence pack. Это не заменяет cryptographic signing, но является хорошим publication-ready уровнем для портфолио.

19. Release decision logic

Policy gate intentionally does not mean “every scanner returns zero”. Gate means: agreed blocking conditions are absent.

Blocking examples:

- Semgrep ERROR findings.
- Bandit HIGH findings.
- Gitleaks findings.
- Public ingress in fixed IaC.
- Trivy HIGH/CRITICAL filesystem vulnerabilities.
- Trivy Dockerfile HIGH/CRITICAL misconfigurations.
- Gype HIGH/CRITICAL SBOM matches.

Non-blocking examples:

- Residual dependency findings that are not HIGH/CRITICAL in configured gate logic.
- Checkov informational/design findings that do not reintroduce public ingress.
- Tool-specific noise that should be triaged, not hidden.

This distinction is important for real DevSecOps work. A poor gate blocks everything and gets bypassed. A useful gate blocks what the team has agreed is unacceptable for release.

20. Reviewer checklist

A reviewer can validate the project without trusting the author:

1. Open PROJECT_TREE.txt and verify repository structure.
2. Open app/vulnerable-version and app/fixed-version.
3. Compare docker/vulnerable.Dockerfile and docker/fixed.Dockerfile.
4. Compare iac/vulnerable/main.tf and iac/fixed/main.tf.
5. Open Phase 6 scan reports and confirm vulnerable findings.
6. Open DEVSECOPS_07_OUTPUT_policy_gate_failed.txt and confirm POLICY_GATE_STATUS=FAILED.
7. Open Phase 8 remediation matrix.
8. Open Phase 9 fixed scan reports.
9. Open DEVSECOPS_09_OUTPUT_policy_gate_passed.txt and confirm POLICY_GATE_STATUS=PASSED.
10. Verify evidence/DEVSECOPS_10_SHA256SUMS.txt.

21. Interview Q&A

Почему не просто один VM?

Потому что один VM смешивает development trust zone, security tooling cache, internet access и evidence generation. Qubes позволяет явно разделить роли.

Почему у fixed версии остались некоторые findings?

Потому что gate design отделяет blocking risk от residual backlog. Остаточные findings сохранены, а не скрыты.

Почему evidence так много?

Потому что проект сделан как defensible case. В реальном audit/incident/security review важны не только результаты, но и доказательства происхождения результатов.

Что самое сильное в проекте?

Связка vulnerable baseline -> failed gate -> remediation -> fixed scans -> passed gate. Это показывает engineering process, а не только знание отдельных tools.

Что можно улучшить дальше?

Добавить CI workflow, signed releases, GitHub branch protection, SARIF upload, container image build, dependency lockfiles и automated report generation.

22. Future improvements

Следующая версия проекта может включать:

- GitHub Actions workflow для автоматического запуска policy gate.
- SARIF output для Semgrep и CodeQL-style review.
- Dependency lockfiles и reproducible Python environment.
- Container image build and image scan.
- Cosign signing for release artifacts.
- SLSA-style provenance notes.
- Renovate/Dependabot simulation.
- Makefile для локального запуска фаз.
- Автоматическую сборку PDF/HTML reports.
- Release notes с GitHub tag artifacts.

23. Final reviewer statement

На момент финальной сборки проект содержит все элементы сильного DevSecOps portfolio case: понятную архитектуру, намеренно уязвимый baseline, реальные scanner outputs, policy decision, remediation, fixed evidence, reports, presentation, hashes and publication cleanup.

Главный результат: FAILED для baseline и PASSED для fixed version подтверждены артефактами, а не заявлены словами.

24. Детальная хронология фаз

Фаза	Название	Что сделано	Тип ценности
00	Project skeleton	Repository directories, evidence folders, branch/tags	foundation
01	Qubes workflow	prepared dev-workbench and security-runner separation documented	architecture
02	Template/AppVM preparation	Tool installation strategy clarified	operations

Фаза	Название	Что сделано	Тип ценности
03	Security tools	Versions and local tool cache captured	toolchain
04	Vulnerable baseline	Application, Dockerfile and IaC vulnerable examples created	baseline
05	Security tooling docs	RU/EN tool usage notes added	documentation
06	Vulnerable scans	All scanner reports generated for baseline	assessment
07	Failed gate	Baseline blocked by policy gate	release control
08	Remediation	Fixed application, Dockerfile and IaC created	engineering fix
09	Fixed scans	Repeat scans and passed policy gate captured	verification
10	Evidence pack	Inventory, project tree and SHA256 manifest created	auditability
11	Report	Bilingual technical report created	communication
12	Presentation	Storyboard, slide source and HTML presentation created	defense
13	Publication cleanup	Backup/cache cleanup, final publication review	release readiness

25. Risk acceptance record

Не все residual findings должны блокировать публикацию лабораторного проекта. Ниже зафиксирована явная позиция по остаточным пунктам.

Residual item	Решение	Почему допустимо	Следующий шаг
Residual pip-audit findings	Documented as non-blocking backlog	No HIGH/CRITICAL gate blocker in configured policy	Keep visible in evidence and track in future CI
Checkov CKV2_AWS_5	Accepted for lab Terraform example	No public ingress remains	Attach SG to real resource in production IaC

Residual item	Решение	Почему допустимо	Следующий шаг
Manual evidence transfer	Accepted Qubes operational model	Explicit boundary crossing is safer than implicit shared state	Automate only with controlled qrexec policy in future
No production deployment	By design	This is a portfolio lab	Add deployment only in a future isolated sandbox

26. Reproduction guide для reviewer

Минимальный путь проверки без запуска всего проекта:

1. Прочитать README.md и PORTFOLIO_SUMMARY.md.
2. Открыть reports/devsecops_lab_report_ru.pdf.
3. Проверить PROJECT_TREE.txt.
4. Проверить vulnerable evidence:
 - evidence/scan_reports/semgrep/DEVSECOPS_06_REPORT_semgrep_vulnerable.json;
 - evidence/scan_reports/bandit/DEVSECOPS_06_REPORT_bandit_vulnerable.json;
 - evidence/scan_reports/gitleaks/DEVSECOPS_06_REPORT_gitleaks_vulnerable.json;
 - evidence/scan_reports/trivy/DEVSECOPS_06_REPORT_trivy_fs_vulnerable.json.
5. Проверить failed gate:
 - evidence/command_outputs/DEVSECOPS_07_OUTPUT_policy_gate_failed.txt.
6. Проверить remediation:
 - evidence/before_after/DEVSECOPS_08_REMEDIATION_MATRIX.csv.
7. Проверить fixed evidence:
 - evidence/scan_reports/semgrep/DEVSECOPS_09_REPORT_semgrep_fixed.json;
 - evidence/scan_reports/bandit/DEVSECOPS_09_REPORT_bandit_fixed.json;
 - evidence/scan_reports/gitleaks/DEVSECOPS_09_REPORT_gitleaks_fixed.json;
 - evidence/scan_reports/trivy/DEVSECOPS_09_REPORT_trivy_fs_fixed.json.
8. Проверить passed gate:
 - evidence/command_outputs/DEVSECOPS_09_OUTPUT_policy_gate_passed.txt.
9. Проверить integrity:
 - evidence/EVIDENCE_INVENTORY.csv;
 - evidence/DEVSECOPS_10_SHA256SUMS.txt.

27. Что именно улучшено в финальной редакции

Финальная редакция была усилена относительно промежуточной версии:

- landing README стал portfolio-ready;
- RU/EN отчеты стали полноценными техническими отчетами;
- добавлены PDF и HTML версии отчетов;
- добавлен final publication review;
- screenshots получили смысловые имена;
- backup директории удалены;
- SHA256 manifest пересобран;

- evidence inventory пересобран;
- release checklist обновлен;
- phase 13 publication readiness документирована.

28. Почему кейс сильный

Кейс сильный по трем причинам.

Первая причина — он показывает процесс, а не набор screenshots. Есть baseline, failure, remediation и passing result.

Вторая причина — он объединяет несколько классов DevSecOps tooling: SAST, SCA, secrets, IaC, container, SBOM и policy gate.

Третья причина — он реализован в Qubes, где separation of duties является частью дизайна, а не декоративной деталью.

29. Expected GitHub reviewer impression

Ожидаемое впечатление от репозитория:

- структура понятна с первого открытия;
- README объясняет историю проекта;
- reports дают executive и technical уровень;
- evidence можно проверить без доверия к автору;
- презентация готова для защиты;
- уязвимая версия не выглядит ошибкой, потому что она явно обозначена как controlled baseline;
- fixed version и policy gate демонстрируют инженерное завершение работы.